

Woxsen University

School of Technology



B.Tech – Computer Science and Engineering
PROJECT BASED LEARNING REPORT

Title of the Project

SWIFTCOURIER: COURIER PARCEL BOOKING AND TRACKING MANAGEMENT SYSTEM

Course Code: 25TU03MJM0

Course Name: Database Management System

Semester: 3

Academic Year: 2026–27

Submitted by:

A Srestadeep – 25WU0101001

Abhigyan Gogoi – 25WU0101004

Jogi Dhanush – 25WU0101053

Section: CSE TIGERS

Under the Guidance of:

Faculty Name: Dr. Kiran Mayee Adavala

Designation: Professor

Department of Computer Science and Engineering

October, 2026

CERTIFICATE

This is to certify that the Project titled “**SwiftCourier: Courier Parcel Booking and Tracking Management System**” is a bonafide work carried out by A Srestadeep (25WU0101001), Abhigyan Gogoi (25WU0101004) and Jogi Dhanush (25WU0101053), B.Tech – Computer Science and Engineering, during the Academic Year 2026–27, in partial fulfillment of the requirements of the course Database Management System (25TU03MJM0), Semester 3, at the School of Technology, Woxsen University, under my supervision and guidance.

To the best of my knowledge, the work embodied in this report has not been submitted elsewhere for the award of any degree, diploma or other academic qualification.

Guide Signature

Dr. Kiran Mayee Adavala

Professor

Department of Computer Science and Engineering

School of Technology, Woxsen University

DECLARATION

We hereby declare that the Project titled “**SwiftCourier: Courier Parcel Booking and Tracking Management System**” submitted by us is an original work carried out under the guidance of Dr. Kiran Mayee Adavala, Professor, Department of Computer Science and Engineering, School of Technology, Woxsen University.

We confirm that this work has not been submitted in part or full for the award of any degree, diploma or other academic qualification in this or any other institution.

All sources of information used in this project have been properly acknowledged and cited as per university referencing standards. We further declare that the plagiarism percentage is within the permissible limit prescribed by the institution.

Place: _____

Date: _____

Signature of the Candidates

A Srestadeep
25WU0101001

Abhigyan Gogoi
25WU0101004

Jogi Dhanush
25WU0101053

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who helped us complete this project, SwiftCourier: Courier Parcel Booking and Tracking Management System.

First and foremost, we thank our guide, Dr. Kiran Mayee Adavala, Professor, Department of Computer Science and Engineering, for her guidance, encouragement and valuable feedback throughout the Database Management System course. Her emphasis on sound database design, normalisation and the correct use of constraints shaped every part of this work.

We are grateful to the Department of Computer Science and Engineering and the School of Technology, Woxsen University, for providing the facilities, software and learning environment needed to design, build and test the system.

We also thank the developers and maintainers of the free and open-source tools on which the project is built, including PHP, MariaDB, Apache, XAMPP, Bootstrap and the qrcode-generator library, which allowed the complete system to be developed at no cost.

Finally, we thank our classmates for their suggestions during testing, and our families for their constant support and encouragement.

A Srestadeep – 25WU0101001
Abhigyan Gogoi – 25WU0101004
Jogi Dhanush – 25WU0101053

ABSTRACT

Small and medium-sized courier operators still rely on paper registers, spreadsheets and phone calls to book parcels and to tell customers where their parcels are. Records get duplicated, charges are worked out by hand, and when a status is overwritten the earlier steps of the parcel's journey are lost.

SwiftCourier is a database-driven web application that solves this problem. It is built with PHP, MySQL, HTML, CSS, Bootstrap and JavaScript and runs on XAMPP. The database, *courier_management*, has four related tables: users, parcels, tracking_history and payments. Customers can register, log in and book parcels; the system calculates the delivery charge, gives every parcel a unique tracking ID and creates a QR code for it. Administrators update the status, location and remarks of each parcel, and every update is saved as a new history record, so the full journey is always kept. Anyone can track a parcel without logging in, either by typing the tracking ID or by scanning the QR code with a phone. The admin dashboard shows live statistics calculated from the database.

The database design was normalised and protected with primary keys, foreign keys and other constraints. All 44 functional and access tests passed, invalid data was rejected by the database, 50 bookings made at the same time all received different tracking IDs, and the QR code was successfully scanned with a smartphone.

Keywords: Database Management System, MySQL, PHP, Courier Management, Parcel Tracking, QR Code, Normalisation

TABLE OF CONTENTS

CERTIFICATE	ii
DECLARATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF FIGURES	vii
LIST OF TABLES	viii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Scope and Organisation of the Report	2
CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES	3
2.1 Problem Definition	3
2.2 Objectives	3
2.3 Requirements	4
2.4 Users and Assumptions	4
2.5 Software and Hardware Requirements	5
CHAPTER 3: LITERATURE REVIEW	6
3.1 Existing Approaches	6
3.2 Related Work	6
3.3 Comparison and Research Gap	7
CHAPTER 4: METHODOLOGY AND SYSTEM DESIGN	8
4.1 Development Approach	8
4.2 System Architecture	8
4.3 How the System Works	9
4.4 Main Rules of the System	10
4.5 Tools and Technologies	10
4.6 Use-Case Design	11
4.7 Database Design	11
CHAPTER 5: IMPLEMENTATION	16
5.1 Development Environment	16
5.2 Database Creation	16
5.3 SQL Commands and Sample Outputs	16
5.4 Login and Registration	17
5.5 Parcel Booking	17
5.6 Tracking and QR Code	18
5.7 Administrator Module	19
5.8 Security	20
CHAPTER 6: RESULTS, TESTING AND DISCUSSION	21
6.1 Functional Testing	21
6.2 Database Constraint Tests	22
6.3 Performance	22
6.4 Queries and Outputs	23
6.5 Changes from the Website Reflected in the Database	25
6.6 Comparison with Objectives	25
CHAPTER 7: CONCLUSION AND FUTURE SCOPE	26
7.1 Conclusion	26
7.2 Learning Outcomes	26
7.3 Limitations	26
7.4 Future Scope	26
7.5 Contribution of Each Member	27
7.6 GitHub Repository	27
REFERENCES	28

LIST OF FIGURES

Figure 4.1: System architecture of SwiftCourier	8
Figure 4.2: Parcel booking process	9
Figure 4.3: Use-case diagram	11
Figure 4.4: ER diagram of the courier_management database	12
Figure 5.1: Booking form and booking confirmation with QR code	18
Figure 5.2: Tracking page on a phone after scanning the QR code	19
Figure 5.3: Administrator dashboard and status update page	20
Figure 6.1: Parcels table before and after booking a parcel on the website	25
Figure 6.2: Deleting a parcel on the website and the database after the delete	25

LIST OF TABLES

Table 2.1: Functional requirements	4
Table 2.2: Software and hardware requirements	5
Table 3.1: Comparison of existing approaches with SwiftCourier	7
Table 4.1: Tools and technologies	10
Table 4.2: Database tables	12
Table 4.3: Data dictionary: users	13
Table 4.4: Data dictionary: parcels	13
Table 4.5: Data dictionary: tracking_history	14
Table 4.6: Data dictionary: payments	14
Table 6.1: Functional test results	21
Table 6.2: Constraint tests	22
Table 6.3: Average query times	22
Table 6.4: Output of Query 1	23
Table 6.5: Output of Query 2	23
Table 6.6: Output of Query 3	24
Table 6.7: Output of Query 4	24
Table 6.8: Achievement of objectives	25
Table 7.1: Contribution of each member	27

CHAPTER 1: INTRODUCTION

Courier services deliver documents and goods every day. For every parcel, a courier company must record who sent it, who will receive it, how heavy it is, how much was charged and where it is at any moment. Keeping this information correct and easy to find is a database problem. This project, SwiftCourier, builds a database for courier booking and tracking, together with a simple website that customers, staff and the public can use.

1.1 Background

A relational database stores information in tables that are linked to each other through keys [1]. Each piece of information is stored only once, and the database itself checks that the data stays correct [2], [3]. This makes relational databases a natural fit for a courier business, where customers, parcels, parcel movements and payments are all connected.

Many small courier counters, however, still use paper registers or spreadsheets. The same parcel may be written down twice with different details, the current status is known only to the staff member who last updated it, and when a status is changed the previous one is lost. Large courier companies have their own tracking systems, but these are expensive and not available to small operators. At the same time, everything needed to build a proper system is free: XAMPP provides PHP and MySQL on any laptop [4], and every smartphone can read QR codes [5].

1.2 Motivation

- **Easy tracking:** customers should be able to see where their parcel is without calling the branch.
- **Complete history:** every movement of a parcel should be saved, not overwritten.
- **Correct charges:** the delivery charge should be calculated by the system using one clear rule.
- **Reliable data:** the database should reject wrong data, such as a duplicate account or a parcel with an impossible weight.
- **Learning:** the project applies the main topics of the course, such as ER diagrams, normalisation, constraints, joins and transactions, in a working system.

The project was built at zero cost, using only free software, so that it can run on any laptop.

1.3 Scope and Organisation of the Report

The system has three types of users: customers, administrators (staff) and public visitors. It covers registration and login, parcel booking with automatic charges, tracking IDs, QR codes, simulated payments, public tracking, status updates, search and filtering, an admin dashboard, and deleting records. Online payment gateways, GPS tracking and SMS alerts are left for future work.

Chapter 2 describes the problem and objectives, Chapter 3 reviews related work, Chapter 4 explains the design, Chapter 5 describes the implementation, Chapter 6 presents the results, and Chapter 7 gives the conclusion and future scope.

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Definition

The aim is to design a database, and a website that uses it, to record courier bookings and the full movement history of every parcel, keep the data correct at all times, and make it available to customers, staff and the public.

A single table with one row per booking and columns such as status_1, status_2 and status_3 does not work: it cannot hold an unknown number of movements, it repeats customer details on every row, and it loses information when a status is changed [2]. Some operations also involve more than one table: a booking must save the parcel, its first tracking record and its payment together, and a status change must update the parcel and add a history record together. If only part of such an operation is saved, the data becomes wrong. Tracking IDs must also stay unique even when two customers book at the same moment.

2.2 Objectives

1. Design a normalised database for users, parcels, tracking history and payments, starting from an ER diagram.
2. Use database constraints (primary keys, foreign keys, unique, not null and check) so that wrong data is rejected.
3. Let customers book parcels online, with the charge calculated automatically and a unique tracking ID generated.
4. Keep a complete tracking history, where every status change adds a new record instead of overwriting the old one.
5. Provide public tracking and working QR codes that open the tracking page on any phone.
6. Give administrators a dashboard with live statistics, search, filtering and delete options.
7. Keep the system secure with hashed passwords, login sessions and protection against common attacks.

2.3 Requirements

Table 2.1: Functional requirements

ID	Requirement	User
FR1	Register, log in and log out; duplicate emails are not allowed.	Customer, Admin
FR2	Book a parcel with sender, receiver, parcel type, weight, delivery date and payment method.	Customer
FR3	Calculate the charge, generate a unique tracking ID and create a QR code.	System
FR4	View, search and filter own parcels, with details and tracking history.	Customer
FR5	Track any parcel by tracking ID or QR code without logging in.	Public
FR6	View dashboard statistics; search and filter all parcels; view customers.	Admin
FR7	Update a parcel's status, location and remarks.	Admin
FR8	Delete a parcel, or a customer who has no parcels.	Admin

The system should also be secure, easy to use on both laptops and phones, fast, and easy to install by importing a single SQL file in phpMyAdmin.

2.4 Users and Assumptions

Customers log in to book parcels and view their own parcels. **Administrators** log in to manage all parcels and customers. **Public visitors** do not log in; they can only track a parcel and see its status and history.

- The system runs on a local XAMPP server; to scan a QR code, the phone and laptop must be on the same Wi-Fi network.
- Payments are simulated: online payments are marked Paid, and Cash on Pickup stays Pending until the parcel is delivered.
- Parcel locations are typed by staff; there is no GPS.

2.5 Software and Hardware Requirements

Table 2.2: Software and hardware requirements

Type	Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or higher
RAM	4 GB minimum (8 GB recommended)
Storage	About 1 GB free space for XAMPP and the project
Network	Wi-Fi, so a phone can open the tracking page by scanning the QR code
Operating system	Windows 10/11 (also works on Linux or macOS)
Server package	XAMPP 8.2: Apache web server, PHP 8.2, MariaDB 10.4 (MySQL)
Database tool	phpMyAdmin (included with XAMPP)
Browser	Google Chrome, Microsoft Edge or Firefox
Code editor	Visual Studio Code
Mobile device	Any smartphone with a camera, for scanning QR codes

CHAPTER 3: LITERATURE REVIEW

This chapter looks at how courier records are kept today and at the published work and standards that guided the design of SwiftCourier.

3.1 Existing Approaches

Paper registers are cheap and simple, but they cannot be searched or shared, and nothing prevents mistakes. **Spreadsheets** can be searched, but they usually keep everything in one large sheet, repeat customer details on every row, overwrite the status as the parcel moves, and allow duplicate or invalid entries. **Commercial courier systems** used by large companies offer online booking, tracking numbers and customer websites, but they are expensive, closed and not available to small operators.

3.2 Related Work

Database design. Codd’s relational model introduced the idea of storing data in linked tables identified by keys [1], and Chen’s entity–relationship (ER) model gave a clear way to draw entities and relationships before creating tables [6]. Standard textbooks explain how to turn an ER diagram into tables [2], [3]. SwiftCourier was designed in exactly this way.

Normalisation. Codd defined the normal forms used to remove repeated data [7], [8]. Kent explained them simply as “every column should depend on the key, the whole key and nothing but the key” [9], and Date notes that a small amount of planned repetition can be kept for speed [10]. SwiftCourier’s tables follow these rules.

Transactions. Haerder and Reuter described the ACID properties, which ensure that a group of database changes is saved completely or not at all [11], [12]. MySQL supports this, and also supports locking rows while they are being read [13] and unique keys that block duplicates [14]. SwiftCourier uses transactions for bookings and status updates.

Web security. Halfond and others showed that SQL injection is one of the most serious attacks on database websites and that prepared statements are the best protection [15]. The OWASP guides recommend prepared statements [16] and security tokens on forms [17], and PHP provides safe password hashing [18]. SwiftCourier follows all of these.

Parcel identification. Couriers label parcels with barcodes, QR codes or RFID tags. QR codes follow the ISO/IEC 18004 standard and can be read by any phone camera [5], while RFID needs special readers [19]. SwiftCourier uses QR codes, generated with a free JavaScript library [20].

3.3 Comparison and Research Gap

Table 3.1: Comparison of existing approaches with SwiftCourier

Feature	Paper register	Spreadsheet	Commercial system	SwiftCourier
Search and reports	No	Limited	Yes	Yes
Full parcel history	No	No	Yes	Yes
Automatic charges	No	Manual	Yes	Yes
Rejects wrong data	No	No	Yes	Yes
Public tracking / QR	No	No	Yes	Yes
Free and open	Yes	Yes	No	Yes

The comparison shows that small courier operators need a system with the main features of a commercial platform, but that is free and simple. SwiftCourier provides this by combining a well-designed database, automatic checks, a complete parcel history and QR-based tracking, using only free software.

CHAPTER 4: METHODOLOGY AND SYSTEM DESIGN

4.1 Development Approach

The database was designed first. The requirements were drawn as an ER diagram, turned into tables and normalised, and the SQL script was tested in phpMyAdmin with sample data before the website was built. The website was then developed step by step (login, booking, tracking and QR codes, the admin pages and finally delete options), and it was tested after each step.

4.2 System Architecture

SwiftCourier has three layers (Figure 4.1). The **browser** shows the pages on a laptop or phone. The **PHP application**, running on Apache, handles the logic of each page. The **MySQL database** stores all the data. The website talks to the database only through prepared SQL statements.

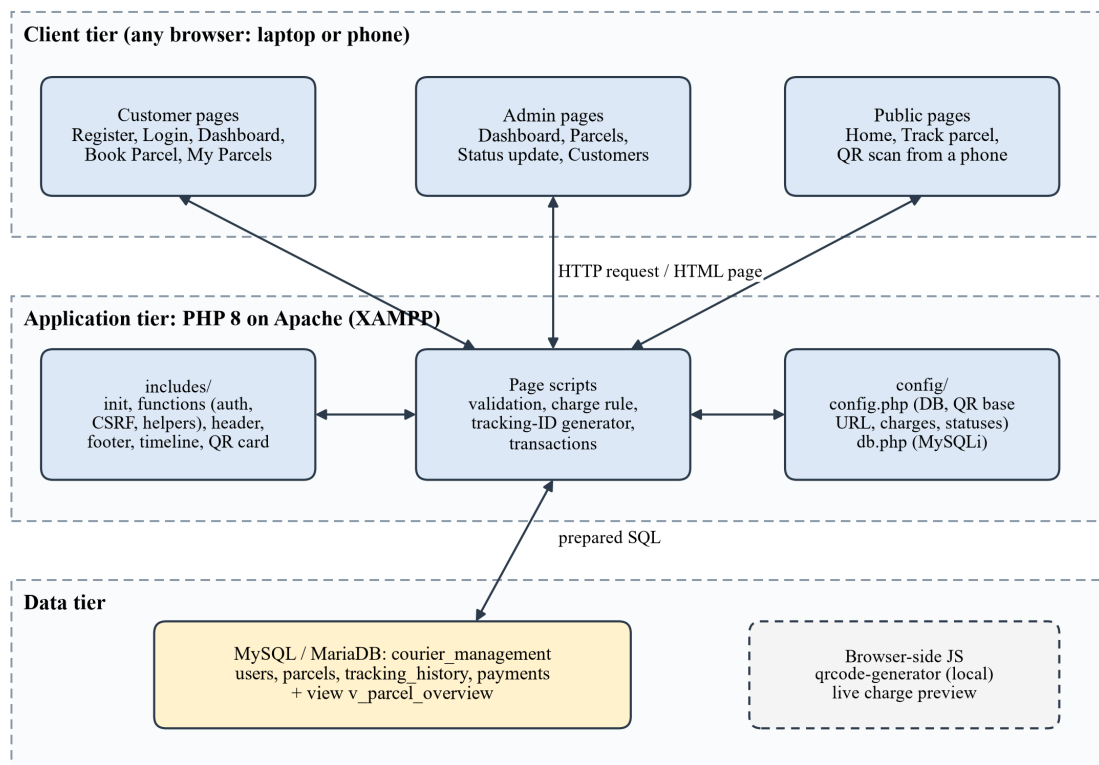


Figure 4.1: System architecture of SwiftCourier

4.3 How the System Works

1. A customer logs in and books a parcel.
2. The system checks the details, calculates the charge and creates a unique tracking ID.
3. The parcel, its first tracking record (“Booked”) and its payment are saved together.
4. The customer sees a confirmation with the tracking ID and a QR code.
5. An administrator updates the parcel’s status and location as it moves; each update adds a new tracking record.
6. Anyone with the tracking ID or the QR code can see the latest status and the full history.

Figure 4.2 shows the booking process. If any step fails, nothing is saved, so the database never contains half a booking.

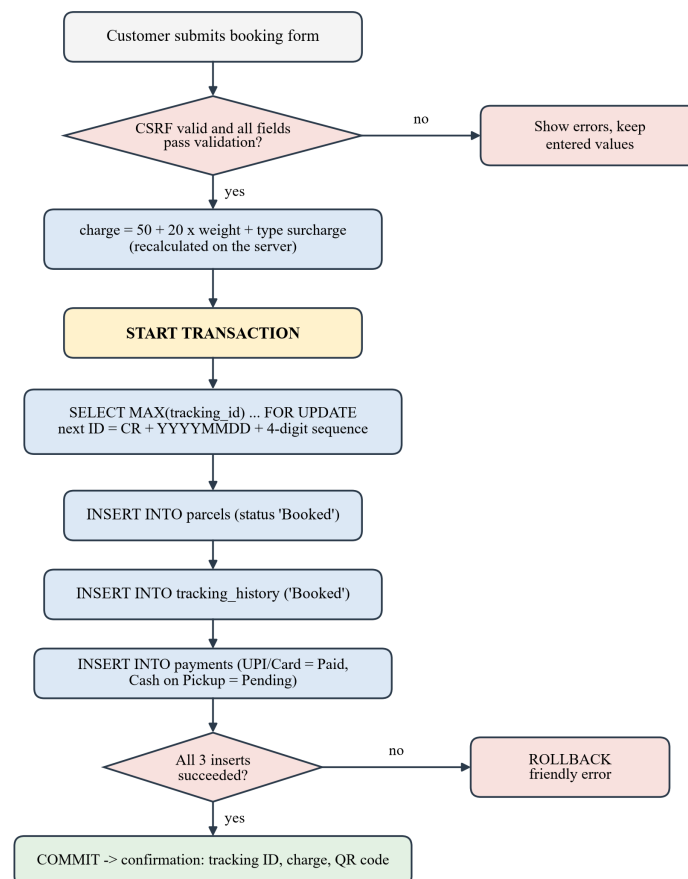


Figure 4.2: Parcel booking process

4.4 Main Rules of the System

- **Delivery charge:** ₹50 base charge + ₹20 per kg + an extra ₹30 for Electronics or ₹40 for Fragile parcels. For example, a 2.5 kg Fragile parcel costs $50 + 50 + 40 = ₹140$. The charge is always recalculated on the server.
- **Tracking ID:** “CR” + the date + a 4-digit number for that day, for example CR202610050001. The next number is read while the rows are locked, and the database also keeps tracking IDs unique, so two parcels can never get the same ID.
- **Status order:** Booked → Picked Up → In Transit → Out for Delivery → Delivered. A parcel can also become Failed and then be re-attempted or Returned. Delivered and Returned parcels cannot be changed, and the admin can only choose a valid next status.
- **QR code:** the QR code contains the link to the parcel’s tracking page. When the site is opened on the laptop, the link uses the laptop’s Wi-Fi address so that a phone can open it.

4.5 Tools and Technologies

Table 4.1: Tools and technologies

Area	Technology
Database	MySQL / MariaDB (through XAMPP [4]), managed with phpMyAdmin
Server-side	PHP 8 on the Apache web server
Front end	HTML, CSS, Bootstrap 5 [21] and JavaScript
QR codes	qrcode-generator (free JavaScript library)
Testing	Automated test scripts and a web browser

4.6 Use-Case Design

Figure 4.3 shows what each type of user can do. Customers register, book and manage their parcels; public visitors track parcels; and administrators view statistics, update parcels, view customers and delete records.

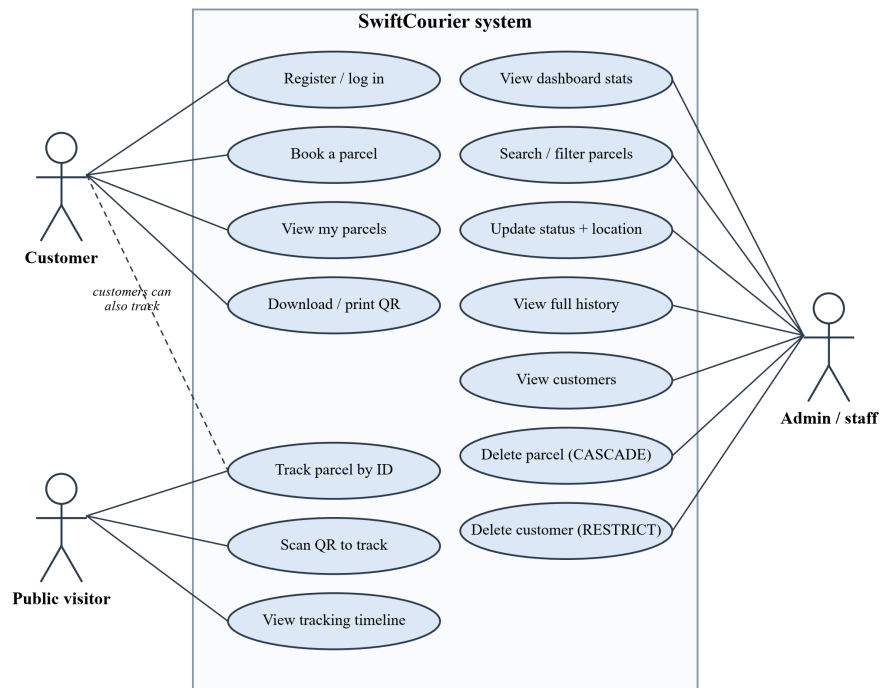


Figure 4.3: Use-case diagram

4.7 Database Design

4.7.1 ER Diagram

The ER diagram (Figure 4.4) has four entities. A **user** books many **parcels**. Each parcel has many **tracking history** records and one or more **payments**. An administrator (also a user) records each tracking update. All four relationships are one-to-many.

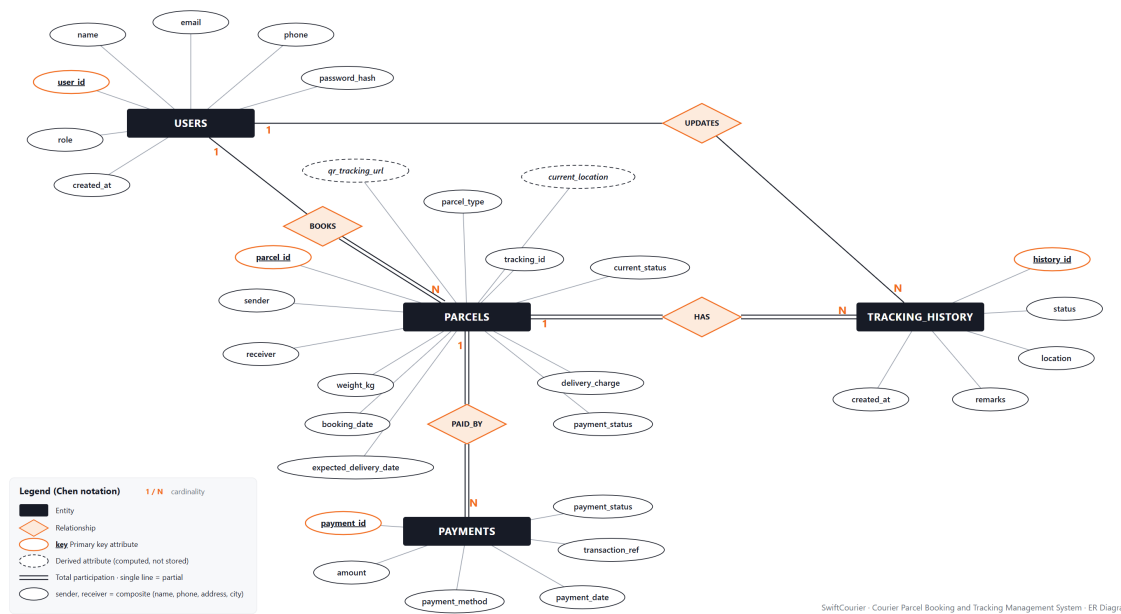


Figure 4.4: ER diagram of the courier_management database

4.7.2 Tables

Each entity became a table with its own primary key, and each relationship became a foreign key. Table 4.2 lists the tables.

Table 4.2: Database tables

Table	Main columns	Keys
users	name, email, phone, password (hashed), role (customer or admin)	Primary key user_id; email is unique
parcels	tracking ID, sender and receiver details, parcel type, weight, booking date, expected delivery date, charge, payment status, current status	Primary key parcel_id; tracking ID is unique; customer_id links to users
tracking_history	status, location, remarks, updated by, date and time	Primary key history_id; parcel_id links to parcels
payments	amount, payment method, payment status, transaction reference, payment date	Primary key payment_id; parcel_id links to parcels; transaction reference is unique

The database also checks values: weight must be between 0 and 50 kg, charges and amounts cannot be negative, and statuses, parcel types and payment methods must come from fixed lists. Deleting a parcel automatically deletes its tracking history and payments, while a customer who still has parcels cannot be deleted. A view, v_parcel_overview, shows each parcel with its customer's name and latest location.

4.7.3 Data Dictionary

Tables 4.3 to 4.6 describe every column of the four tables.

Table 4.3: Data dictionary: users

Column	Data type	Constraint	Description
user_id	INT	Primary key, auto increment	Unique user number
name	VARCHAR(100)	Not null	Full name
email	VARCHAR(150)	Not null, unique	Login email
phone	VARCHAR(15)	Not null	Phone number
password_hash	VARCHAR(255)	Not null	Hashed password
role	ENUM	customer or admin	Type of user
created_at	TIMESTAMP	Default: current time	Registration time

Table 4.4: Data dictionary: parcels

Column	Data type	Constraint	Description
parcel_id	INT	Primary key, auto increment	Unique parcel number
tracking_id	CHAR(14)	Not null, unique	Tracking ID, e.g. CR202610050001
customer_id	INT	Foreign key → users	Customer who booked
sender_name, sender_phone, sender_address, sender_city	VARCHAR	Not null	Sender details
receiver_name, receiver_phone, receiver_address, receiver_city	VARCHAR	Not null	Receiver details
parcel_type	ENUM	Document, Package, Electronics, Fragile, Other	Type of parcel
weight_kg	DECIMAL(6,2)	Check: 0 to 50	Weight in kg
booking_date	DATETIME	Default: current time	Booking time
expected_delivery_date	DATE	Not null	Expected delivery
delivery_charge	DECIMAL(10,2)	Check: 0 or more	Delivery charge (₹)
payment_status	ENUM	Pending, Paid, Refunded	Payment summary
current_status	ENUM	Booked ... Returned (7 values)	Latest status

Table 4.5: Data dictionary: tracking_history

Column	Data type	Constraint	Description
history_id	INT	Primary key, auto increment	Unique record number
parcel_id	INT	Foreign key → parcels (cascade)	Parcel this update belongs to
status	ENUM	7 allowed statuses	Status at this update
location	VARCHAR(120)	Not null	Where the parcel was
remarks	VARCHAR(255)	Optional	Note for this update
updated_by	INT	Foreign key → users (set null)	User who made the update
created_at	DATETIME	Default: current time	Time of update

Table 4.6: Data dictionary: payments

Column	Data type	Constraint	Description
payment_id	INT	Primary key, auto increment	Unique payment number
parcel_id	INT	Foreign key → parcels (cascade)	Parcel paid for
amount	DECIMAL(10,2)	Check: 0 or more	Amount (₹)
payment_method	ENUM	Cash on Pickup, UPI, Card, Net Banking	How it was paid
payment_status	ENUM	Pending, Paid, Failed, Refunded	Payment state
transaction_ref	VARCHAR(30)	Not null, unique	Transaction reference
payment_date	DATETIME	Default: current time	Payment time

4.7.4 Normalisation

The design started from a paper-register style table that kept everything in one row, including repeated status columns. Moving each status update into its own row in tracking_history removed the repeated columns (**First Normal Form**). Keeping parcel details in parcels and giving each history record its own key removed partial dependencies (**Second Normal Form**). Moving customer details to users and payment details to payments removed indirect dependencies (**Third Normal Form**). In every table, the only columns that decide other columns are keys, so the design also meets **BCNF** [8].

Two columns in parcels, `current_status` and `payment_status`, are kept on purpose as quick summaries of the latest tracking record and the payment record. They make the dashboard faster, and they are always updated in the same transaction as the records they summarise, so they never disagree.

CHAPTER 5: IMPLEMENTATION

5.1 Development Environment

The system was built on Windows 11 using XAMPP (Apache, PHP 8.2 and MariaDB 10.4) and runs at <http://localhost/courier/>. The project is organised into simple folders: the main pages, an admin folder, a config folder for settings, an includes folder for shared code, an assets folder for styles and scripts, and a database folder containing the SQL script and demonstration queries.

5.2 Database Creation

The whole database is created by importing one file, `courier_management.sql`, in phpMyAdmin. It creates the four tables with all their keys and constraints, the view, and sample data: 6 users, 12 parcels covering every status, 44 tracking records and 12 payments. The sample passwords are stored in hashed form, just like passwords created through the website.

5.3 SQL Commands and Sample Outputs

The main SQL commands used to build the database are shown below with the output MySQL returned.

Creating the database:

```
CREATE DATABASE courier_management;  
USE courier_management;
```

Output: Query OK, 1 row affected

Creating a table (users; the other three tables follow the same pattern):

```
CREATE TABLE users (  
    user_id      INT AUTO_INCREMENT PRIMARY KEY,  
    name         VARCHAR(100) NOT NULL,  
    email        VARCHAR(150) NOT NULL UNIQUE,  
    phone        VARCHAR(15)  NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    role         ENUM('customer','admin') NOT NULL DEFAULT 'customer',  
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Output: Query OK, 0 rows affected

A table with a foreign key (tracking_history):

```
CREATE TABLE tracking_history (  
  history_id INT AUTO_INCREMENT PRIMARY KEY,  
  parcel_id INT NOT NULL,  
  status ENUM('Booked','Picked Up','In Transit','Out for Delivery',  
              'Delivered','Failed','Returned') NOT NULL,  
  location VARCHAR(120) NOT NULL,  
  remarks VARCHAR(255),  
  updated_by INT,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (parcel_id) REFERENCES parcels(parcel_id) ON DELETE CASCADE  
);
```

Output: Query OK, 0 rows affected

Inserting a record:

```
INSERT INTO users (name, email, phone, password_hash, role)  
VALUES ('Kiran Kumar', 'kiran@example.com', '9876500077', '<hashed>', 'customer');
```

Output: Query OK, 1 row affected

The sample data contains 6 users, 12 parcels, 44 tracking records and 12 payments, so every table has at least 5 records.

5.4 Login and Registration

When a user registers, every field is checked, duplicate emails are rejected, and the password is stored only as a secure hash [18]. At login, the password is checked against the hash and a new session is started. Every page checks whether the user is logged in and has the right role, and customers can only open their own parcels.

5.5 Parcel Booking

The booking form collects the sender and receiver details, parcel type, weight, expected delivery date and payment method, and shows the charge as the customer types. When the form is submitted, the server checks every field again and recalculates the charge. It then saves the parcel, its first tracking record and its payment in one transaction: if anything fails, nothing is saved. The customer then sees a confirmation page with the tracking ID, charge, payment details and QR code (Figure 5.1).

The image displays two mockups of the Swift Courier web application. The left mockup shows the 'Book a Parcel' form, which includes sections for 'Sender details' (Name: Rahul Sharma, Phone: 9876543210, Address: Flat 302, Green Residency, Mathpur, Hyderabad), 'Receiver details' (Name: Meena Iyer, Phone: 9830012345, Address: 14, Park Street, Kolkata), and 'Parcel details' (Type: Fragile, Weight: 2.5 kg, Expected delivery: 10-10-2026). The right mockup shows the 'Charges' summary (Base charge: ₹150.00, 2.5 kg + ₹20, Type surcharge: ₹40.00, Total: ₹140.00), a 'Booking confirmed' message with tracking ID CR202618060001, and a 'Parcel QR code' for tracking.

5.6 Tracking and QR Code

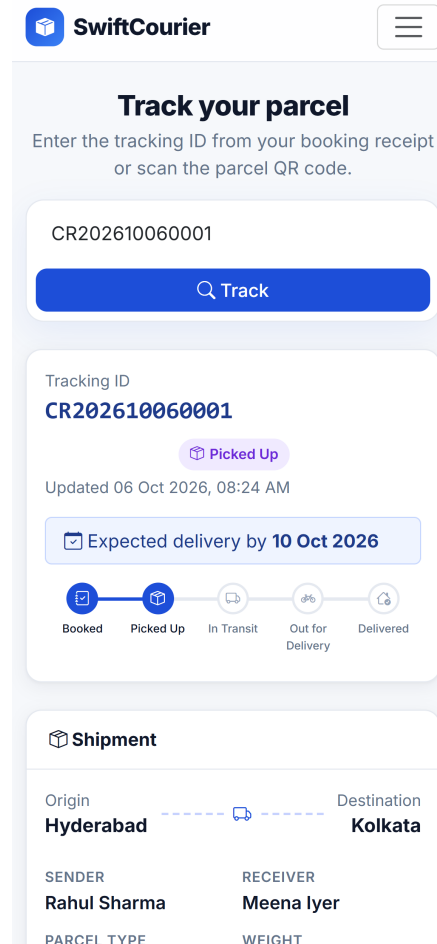


Figure 5.2: Tracking page on a phone after scanning the QR code

5.7 Administrator Module

The admin dashboard shows the total number of customers and parcels, the number of parcels in each status, total revenue, recent bookings, top customers and the latest updates, all calculated with SQL queries such as COUNT, SUM, GROUP BY and JOIN (Figure 5.3). Administrators can search parcels by tracking ID, name or phone number, filter them by status, and view all customers with their number of bookings.

To update a parcel, the administrator chooses the next status and enters the location and remarks. The system checks that the change is allowed, then updates the parcel and adds a new tracking record in the same transaction. When a Cash on Pickup parcel is delivered, its payment is marked as Paid. Administrators can also delete a parcel, which removes its history and payments automatically, or delete a customer, which the database blocks if that customer still has parcels.

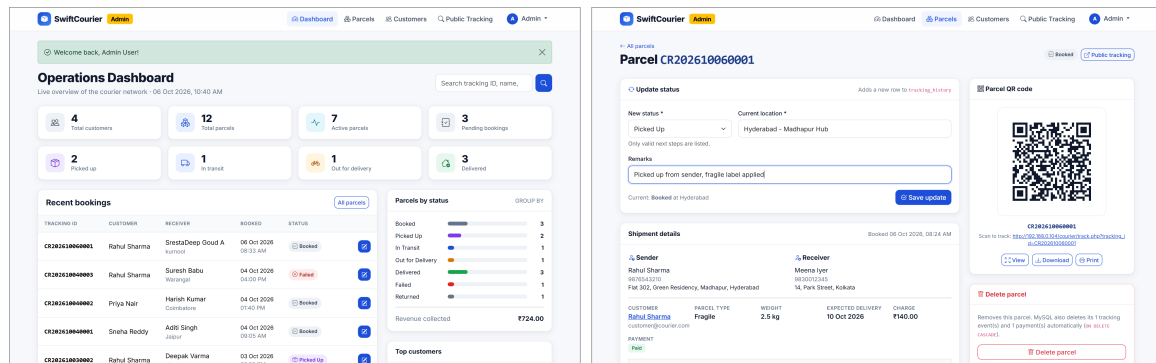


Figure 5.3: Administrator dashboard and status update page

5.8 Security

- All database queries use prepared statements [22], which prevents SQL injection [16].
- Passwords are stored as hashes, never as plain text.
- Every form includes a security token to block forged requests [17].
- Admin pages can only be opened by administrators, and customers only see their own parcels.
- All output is escaped to prevent script injection, and errors are logged instead of being shown to users.

CHAPTER 6: RESULTS, TESTING AND DISCUSSION

6.1 Functional Testing

The website was tested with an automated script that logs in as a customer and as an administrator and performs every main action. The database was reset before testing. All 44 checks passed. Table 6.1 summarises the main test cases.

Table 6.1: Functional test results

#	Test case	Expected result	Result
1	Track a parcel with a valid, an invalid and an unknown tracking ID	Correct details; error messages	Pass
2	Register a new user; register with an existing email	Account created; duplicate rejected	Pass
3	Log in with a wrong password, then the correct one	Error, then dashboard	Pass
4	Customer opens another customer's parcel	Access denied	Pass
5	Book a 2.5 kg Fragile parcel	Tracking ID created; charge ₹140	Pass
6	Book with invalid data	Errors shown; nothing saved	Pass
7	Admin updates status; public page checked	New status shown immediately	Pass
8	Invalid status change (e.g. Booked → Delivered)	Rejected	Pass
9	Deliver a Cash on Pickup parcel	Payment becomes Paid	Pass
10	Delete a parcel; delete a customer with parcels	History removed; customer delete blocked	Pass
11	Open admin pages without logging in	Redirected to login	Pass

QR code test. The QR code was scanned with a team member's smartphone on the same Wi-Fi network. It opened the parcel's tracking page and showed its current status and history directly from the database.

6.2 Database Constraint Tests

To prove that the database itself protects the data, wrong data was inserted directly in MySQL. Every attempt was rejected with an error from the database (Table 6.2) [23].

Table 6.2: Constraint tests

Wrong data entered	Result
A parcel weighing 75 kg	Rejected by the CHECK constraint
A parcel for a customer who does not exist	Rejected by the FOREIGN KEY
A second account with the same email	Rejected by the UNIQUE constraint
A parcel with an existing tracking ID	Rejected by the UNIQUE constraint
Deleting a customer who still has parcels	Blocked by the FOREIGN KEY (RESTRICT)
An invalid status such as 'Lost'	Rejected (only listed values are allowed)

The tracking ID rule was also tested by making 50 bookings at the same time from 10 parallel processes. All 50 parcels received different tracking IDs, numbered from 0001 to 0050 without gaps.

6.3 Performance

Each query was run many times on the development laptop and timed (Table 6.3). All queries finished in less than one millisecond, and bookings and status updates in about three milliseconds. Web pages loaded in about 31–44 milliseconds on average, which is fast for users.

Table 6.3: Average query times

Operation	Time (ms)
Count customers	0.23
Count parcels by status (GROUP BY)	0.21
Recent bookings (JOIN)	0.32
Bookings per customer (LEFT JOIN)	0.50
Tracking history of a parcel	0.40
Book a parcel (transaction)	2.77
Update a status (transaction)	3.09

6.4 Queries and Outputs

The following queries were run on the sample data. Each output is exactly what MySQL returned.

Query 1: number of parcels in each status (aggregate function with GROUP BY)

```
SELECT current_status, COUNT(*) AS parcels
FROM parcels
GROUP BY current_status
ORDER BY parcels DESC;
```

Table 6.4: Output of Query 1

current_status	parcels
Delivered	3
Booked	2
Picked Up	2
In Transit	2
Failed	1
Returned	1
Out for Delivery	1

Query 2: recent bookings with customer names (JOIN)

```
SELECT p.tracking_id, u.name AS customer, p.receiver_city, p.current_status
FROM parcels p
JOIN users u ON u.user_id = p.customer_id
ORDER BY p.booking_date DESC
LIMIT 5;
```

Table 6.5: Output of Query 2

tracking_id	customer	receiver_city	current_status
CR202610040003	Rahul Sharma	Warangal	Failed
CR202610040002	Priya Nair	Coimbatore	Booked
CR202610040001	Sneha Reddy	Jaipur	Booked
CR202610030002	Rahul Sharma	Visakhapatnam	Picked Up
CR202610030001	Arjun Mehta	Mumbai	Picked Up

Query 3: bookings and total charges per customer (LEFT JOIN, COUNT, SUM)

```
SELECT u.name, COUNT(p.parcel_id) AS bookings, SUM(p.delivery_charge) AS total_charges
FROM users u
LEFT JOIN parcels p ON p.customer_id = u.user_id
WHERE u.role = 'customer'
GROUP BY u.user_id, u.name
ORDER BY bookings DESC;
```

Table 6.6: Output of Query 3

name	bookings	total_charges
Rahul Sharma	5	500.00
Priya Nair	3	200.00
Arjun Mehta	2	300.00
Sneha Reddy	2	204.00

Query 4: full tracking history of one parcel (JOIN of three tables)

```
SELECT th.created_at, th.status, th.location, u.name AS updated_by
FROM tracking_history th
JOIN parcels p ON p.parcel_id = th.parcel_id
LEFT JOIN users u ON u.user_id = th.updated_by
WHERE p.tracking_id = 'CR202610010001'
ORDER BY th.created_at;
```

Table 6.7: Output of Query 4

created_at	status	location	updated_by
2026-10-01 08:50	Booked	Hyderabad	Rahul Sharma
2026-10-01 13:00	Picked Up	Hyderabad - Madhapur Hub	Admin User
2026-10-02 06:00	In Transit	Hyderabad Sorting Centre	Admin User
2026-10-04 07:30	In Transit	Chennai Sorting Centre	Operations Staff
2026-10-05 09:10	Out for Delivery	Chennai - Besant Nagar	Operations Staff

Revenue from paid payments, calculated with SUM, was ₹780 (UPI ₹300, Card ₹330, Net Banking ₹150).

6.5 Changes from the Website Reflected in the Database

To show that the website really changes the database, the same table was viewed in phpMyAdmin before and after each action. Figure 6.1 shows the parcels table before and after a booking made on the website: the new parcel appears at the top. Figure 6.2 shows a parcel being deleted on the website and the database afterwards: its tracking history was removed automatically by ON DELETE CASCADE. A customer added through the registration page and then deleted from the admin page appeared in and disappeared from the users table in the same way.

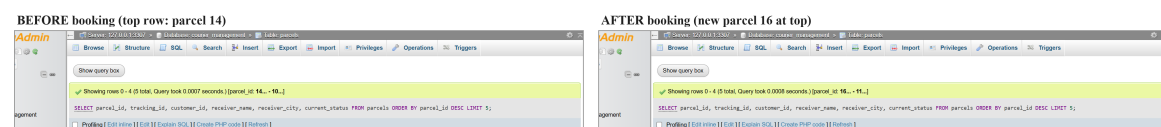


Figure 6.1: Parcels table before and after booking a parcel on the website

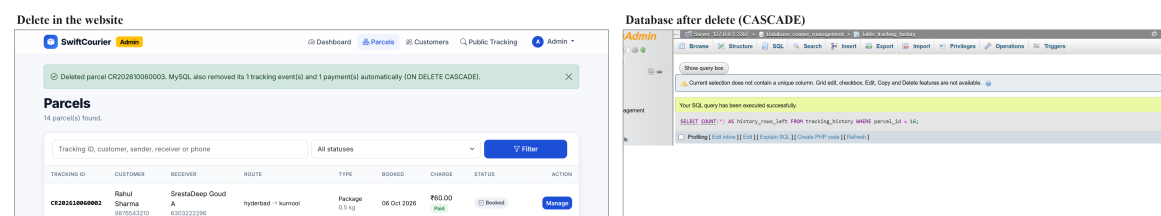


Figure 6.2: Deleting a parcel on the website and the database after the delete

6.6 Comparison with Objectives

Table 6.8: Achievement of objectives

Objective	Status
1. Normalised database design	Achieved
2. Database constraints	Achieved
3. Online booking with unique tracking IDs	Achieved
4. Complete tracking history	Achieved
5. Public tracking and QR codes	Achieved
6. Admin dashboard, search and delete	Achieved
7. Security	Achieved

The results show that when the database is designed well and checks its own data, the website becomes simpler and more reliable. Because every update is added as a new record, the full journey of each parcel is always available. Testing also showed that XAMPP does not reject invalid status values by default, so the system turns on strict checking for its connection to make sure such values are always rejected.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

SwiftCourier is a working courier booking and tracking system built on a well-designed MySQL database. Customers can book parcels and receive a tracking ID and QR code; administrators can update each parcel's journey; and anyone can track a parcel from a phone. The database keeps the data correct through keys and constraints, saves related changes together, and never loses a parcel's history. All tests passed, and the QR code worked on a real phone.

7.2 Learning Outcomes

- Designing an ER diagram and turning it into normalised tables.
- Using primary keys, foreign keys and other constraints to keep data correct.
- Using transactions so that related changes are saved together.
- Writing SQL queries with joins, grouping and aggregate functions.
- Building and testing a secure website connected to a database.

7.3 Limitations

Payments are simulated, parcel locations are typed by staff, the system runs on a local server, and there is only one administrator role.

7.4 Future Scope

1. Add branches, hubs and delivery agents with their own logins.
2. Connect a real online payment gateway.
3. Send SMS or email alerts when a parcel's status changes.
4. Add proof of delivery, such as a photo or OTP.
5. Show live parcel location on a map using GPS.
6. Host the system online so QR codes work from any network.

7.5 Contribution of Each Member

Table 7.1: Contribution of each member

Member	Roll number	Contribution
A Srestadeep	25WU0101001	Database design, table creation, constraints and SQL queries
Abhigyan Gogoi	25WU0101004	ER diagram, normalisation and project report
Jogi Dhanush	25WU0101053	User interface (website) development and testing

7.6 GitHub Repository

The complete project, including the source code, SQL file, ER diagram, screenshots, presentation and this report, is available at:

<https://github.com/akkalasrestadeepgoud333-code/DBMS-Course-Project>

REFERENCES

- [1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.
- [2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.
- [3] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [4] Apache Friends, "XAMPP Apache + MariaDB + PHP + Perl." [Online]. Available: <https://www.apachefriends.org/>
- [5] Information Technology—Automatic Identification and Data Capture Techniques—QR Code Bar Code Symbology Specification, ISO/IEC Standard 18004:2015, International Organization for Standardization, Geneva, Switzerland, 2015.
- [6] P. P.-S. Chen, "The entity-relationship model—Toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.
- [7] E. F. Codd, "Further normalization of the data base relational model," in *Data Base Systems*, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.
- [8] E. F. Codd, "Recent investigations in relational data base systems," in *Proc. IFIP Congress 74*, Stockholm, Sweden, 1974, pp. 1017–1021.
- [9] W. Kent, "A simple guide to five normal forms in relational database theory," *Communications of the ACM*, vol. 26, no. 2, pp. 120–125, Feb. 1983, doi: 10.1145/358024.358054.
- [10] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.
- [11] T. Haerder and A. Reuter, "Principles of transaction-oriented database recovery," *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, Dec. 1983, doi: 10.1145/289.291.
- [12] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA, USA: Morgan Kaufmann, 1993.
- [13] Oracle Corporation, "Locking reads," *MySQL 8.0 Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html>
- [14] Oracle Corporation, "FOREIGN KEY constraints," *MySQL 8.0 Reference Manual*. [Online]. Available:

<https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>

- [15] W. G. J. Halfond, J. Viegas, and A. Orso, "A classification of SQL-injection attacks and countermeasures," in *Proc. IEEE Int. Symp. Secure Software Engineering (ISSSE)*, Arlington, VA, USA, 2006.
- [16] OWASP Foundation, "SQL Injection Prevention Cheat Sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [17] OWASP Foundation, "Cross-Site Request Forgery Prevention Cheat Sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- [18] The PHP Group, "password_hash," *PHP Manual*. [Online]. Available: <https://www.php.net/manual/en/function.password-hash.php>
- [19] K. Finkenzeller, *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards, Radio Frequency Identification and Near-Field Communication*, 3rd ed. Chichester, U.K.: Wiley, 2010.
- [20] K. Arase, "qrcode-generator: QR code generator implementation in JavaScript and other languages," GitHub repository. [Online]. Available: <https://github.com/kazuhiroarase/qrcode-generator>
- [21] Bootstrap Team, "Bootstrap 5.3 documentation." [Online]. Available: <https://getbootstrap.com/docs/5.3/>
- [22] The PHP Group, "Prepared statements," *PHP Manual: MySQL Improved Extension*. [Online]. Available: <https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php>
- [23] MariaDB Foundation, "CONSTRAINT," *MariaDB Knowledge Base*. [Online]. Available: <https://mariadb.com/kb/en/constraint/>